RIDEY — 앞사람들의 헛걸음이 고장을 알려 준다

공공자전거 대여기록만으로 신고되지 않은 고장 자전거를 찾는 방법

> 한 줄: 사람들이 귀찮아서 안 하는 고장 신고를, 이미 공개된 대여기록 속 "빌리자마자 반납" 의 연쇄가 대신한다.

1. 문제

- 서울 따릉이는 고장 신고가 **매년 10만 대 이상** 접수된다(헤럴드경제, 2026-09). 이용자 불만 1순위는 "고장 자전거가 너무 많다".
- 서울시설공단은 **이용자가 신고해야** 고장을 안다. 하지만 고장 자전거를 만난 사람 대부분은 신고하지 않고 그 자리에서 반납한 뒤 다른 자전거를 빌린다.
- 그 자전거는 앱에 "대여 가능" 으로 남아, 다음 사람이 또 빌렸다가 또 반납한다.

2. 핵심 아이디어

- **헛대여**: 같은 대여소에 **3분 안에, 300m 도 안 가고** 반납한 대여. 타 보려다 포기한 흔적이다.
- 한 번의 헛대여는 우연(마음이 바뀜, 결제 오류)일 수 있다. 그러나 **서로 다른 사람**이 **연달아** 포기했다면 자전거 탓이다.
- 같은 사람이 다시 빌려 본 경우(생년·성별이 같은 연속 대여)는 한 사람으로 센다.
- 헛대여는 **신고 없이, 이용만 해도** 기록에 쌓인다 — 신고를 대신할 수 있는 유일한 신호다.

3. 데이터

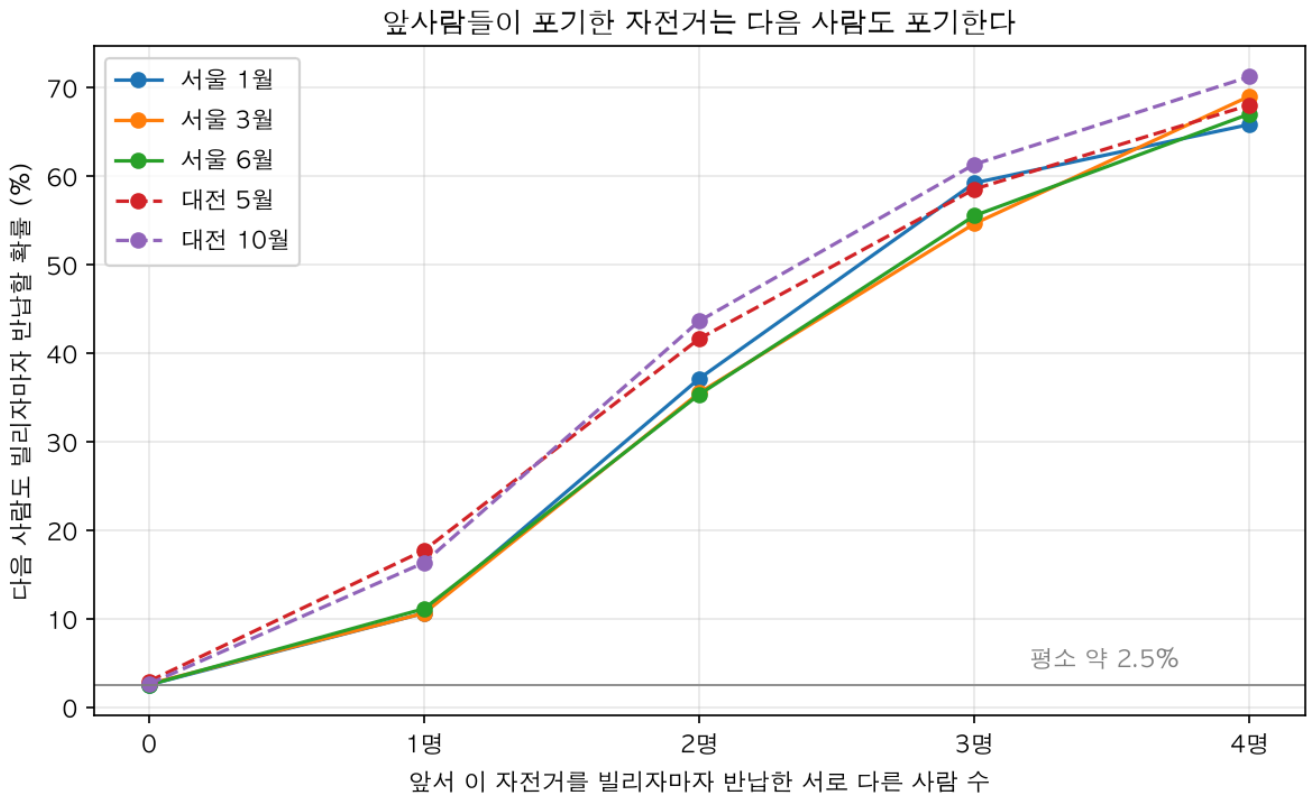
자료	출처	규모
서울 따릉이 대여이력 2026년 1·3·6월	서울 열린데이터광장 OA-15182	8,748,989건
서울 따릉이 고장신고 2026년 1~6월	서울 열린데이터광장 OA-15644	자전거번호·일시·종류
서울 대여소 정보	서울 열린데이터광장 OA-13252	2,789곳
대전 타슈 대여이력 2025년 5·10월	공공데이터포털 15137219	1,223,202건
모든 결과는 `python data/download.py` → `python analysis/report.py` 로 누구나 다시 만들 수 있다.		

4. 방법

1. 자전거마다 대여를 시간순으로 늘어놓고 헛대여를 표시한다.
2. 각 대여 직전까지 "서로 다른 사람의 헛대여가 몇 번 이어졌나(연쇄)" 를 센다. 정상 이용이 한 번 나오면 0으로 돌아간다.
3. **경보**: 연쇄가 2가 되는 순간(노랑), 3이면 빨강.
4. **기준을 미리 본 데이터에 맞추지 않기**: 헛대여 시간(60/120/180초)·거리(100/200/300m)·연쇄(2/3회)를 **1월 자료로만** 고르고, 3·6월과 대전에 그대로 적용해 시험했다.

5. 결과

5-1. 앞사람들이 포기할수록 다음 사람도 포기한다



도시·달	대여	앞선 포기 0명 → 다음 사람 포기	1명	2명	3명
서울 2026-01	1,640,582	2.5%	10.6%	**37.1%**	59.2%
서울 2026-03	2,925,610	2.6%	10.6%	**35.5%**	54.6%
서울 2026-06	4,182,797	2.5%	11.1%	**35.3%**	55.5%
대전 2025-05	614,735	2.9%	17.7%	**41.6%**	58.5%
대전 2025-10	608,467	2.6%	16.3%	**43.6%**	61.3%

두 도시·다섯 달에서 같은 모양. 서로 다른 두 사람이 포기한 자전거는 다음 사람도 **약 14배** 더 포기한다.

5-2. 경보가 맞는 비율과 막을 수 있는 헛걸음

1월로 고른 규칙(3분·300m·2연속)을 그대로 적용: 경보 뒤 그 자전거를 빌린 사람이 또 헛걸음한 비율(= 경보 정밀도) 서울 1월 54.1% → 3월 52.1% · 6월 51.8%(차이 ≤2.3%p), 대전 58.5% · 60.2%. 3연속이면 약 70%. ※ 이 숫자는 경보가 켜져 있는 동안 빌린 **모든** 사람 기준이라 긴 연쇄(21명 연속 같은)가 크게 들어간다. **경보 한 건마다 그 뒤 처음 빌린 다른 사람**만 세면 서울 32.1~34.3%(평소 2.5%의 약 13배, `docs/per_alarm.md`) — 실시간 운영 채점은 이 기준으로 비교한다. 막을 수 있던 헛걸음: 서울 하루 **90~243명**(달·계절 따라), 대전 하루 114~132명.

5-3. 신고보다 하루 가까이 빠르다

서울 고장 신고 중 12~23%는 신고 전 7일 안에 경보가 먼저 울렸고, **경보 → 신고 중앙값 20~25시간**, 그 사이 한 자전거에서 평균 **3.5~4.6명**이 헛걸음했다. 또 서로 다른 사람의 연속 헛대여 중 **49~61% 는 7일이 지나도 고장 신고가 없었다** — 신고만 보면 놓치는 고장이다.

5-4. 매일 아침 "신고 안 된 고장 의심 목록"

어제 자정까지의 기록만으로(하루 늦어도):

달	아침 목록	오늘 첫 이용자가 헛걸음	오늘 막을 수 있는 헛걸음	아직 신고 없음
2026-01	하루 46대	42.0%	34명	81.6%
2026-03	하루 60대	35.5%	47명	76.2%
2026-06	하루 79대	33.4%	71명	66.0%

- **운영과 같은 방식으로 7일 연속 예행연습**(`server/rehearse.py` , `docs/phase3_rehearsal.md`): 매일 아침 최근 7일 기록만 받아 목록을 만들고, 다음 날 아침 어제 목록을 채점. 한 달 전체로 만든 목록과 99.6% 같고, 6/8~14 목록 자전거를 그날 처음 빌린 사람 중 **42.3%** 가 헛걸음(평소 약 2.5%). 창을 1일로 줄이면 목록의 18% 가 달라진다.
- 목록은 **어제 마지막 대여가 연쇄로 끝난 자전거만** 넣는다. 연쇄 뒤 2일 넘게 아무도 안 빌린 자전거(하루 약 200대)는 다음 사람 헛걸음이 10~12% 로 뚝 떨어진다 — 이미 수거·수리됐을 가능성이 커서, 넣으면 목록이 세 배로 늘고 정확도는 떨어진다(6월 3주 실측).

5-5. 정비 동선

경보의 25% 가 대여소 5.3% 에, 50% 가 16.2% 에 몰린다. 한강버스 선착장처럼 처음 타 보는 이용자가 많은 곳은 경보 뒤 고장 신고율이 22%(그 외 33%)로 낮아 기준을 한 단계 올린다.

가장 많이 막는 동선 (`engine/route.py` , `docs/route_backtest.md`): 근무 시간 안에 막을 헛걸음이 가장 많은 대여소를 고르고 순서를 정한다(오리엔티어링 문제 — 욕심 삽입 + 2-opt). 대여소 값 = 목록 자전거의 그날 헛걸음 평균(연쇄 길이 × 대여소 붐빔, 1·3월로 맞춤) × 도착 뒤 남은 대여 비율. 6월 되짚기(구·권역마다 기사 한 명, 9시 출발): 시간이 빠듯할 때 순위대로 돌기보다 **12~17% 더** 막는다(구 1시간 429 vs 366명, 권역 3시간 363 vs 315명). 시간이 넉넉하면 차이가 거의 없다(구 3시간 +2%) — 구마다 목록이 평균 5곳이라 다 돌 수 있어서. 미래를 아는 상한은 +79~205% 라, 어느 자전거가 곧 빌려질지 더 잘 알수록 더 막을 여지가 있다.

5-6. 헛대여 모양으로 고장 종류 짐작 (`docs/fault_kind.md`)

경보 뒤 24시간 안에 신고된 종류별로 헛대여를 비교하면 두 무리로 갈린다: **단말기**(중앙값 13초, 71% 가 30초 안 — 잠금이 안 풀려 바로 반납)와 안장(30초·49%)은 "보자마자", 타이어·체인·페달(37~38초, 약 14% 는 50m 넘게 움직였다 돌아옴)은 "타 보고" 포기. **하지만 한 대씩 이름표를 붙이기엔 모자라다** (`docs/fault_kind_per_bike.md`): 신고 전 24시간 헛대여가 모두 빨랐던 자전거에 "단말기" 를 붙이면 맞는 비율이 10~14% 뿐이다(신호는 있음, AUROC 0.69~0.78). 단말기가 신고의 2~4% 로 드물어서다. 평균의 차이와 한 대 판정은 다르다. → 앱에 고장 종류는 표시하지 않고, 종류는 **구조대 3초 확인**(체인·타이어·안장·브레이크·멀쩡함)으로 사람에게 받는다.

5-7. 자체 모델 — "지금 목록의 이 자전거, 다음 사람도 반납할 확률" (`docs/model.md`)

규칙(서로 다른 2명 연달아)은 설명하기 쉽지만, 같은 노랑이어도 지난주에 여러 번 말썽이었던 자전거와 처음인 자전거, 방금 반납된 자전거와 반나절째 서 있는 자전거가 같아 보인다. **그래디언트 부스팅**(나무 60그루·깊이 3)을 배웠다. 특징 7개 — 연쇄, 지난 7일 헛대여·대여 수, 이전 경보 수, 대여 시간, **경과 시간**, **외면**(그사이 같은 대여소에서 남이 빌려 간 수, Kaspi 2016 의 생각). 이용자 정보는 쓰지 않는다.

- 처음엔 반납 **순간** 의 자전거로 배웠더니(같은 수를 고를 때 규칙보다 +2~4%p, `docs/model_event.md`) 지금 목록에선 진짜 고장 수를 크게 잡았다(6월: 기대 39대, 실제 25대). 목록의 자전거는 몇 시간째 서 있고, 오래 서 있을수록 다음 사람 헛대여가 낮다(3시간 안 44% → 12~24시간 22%).
- 그래서 **목록에 올라 있는 동안** 의 표본(3시간마다 목록, 세 달 3만 3천 건)으로 다시 배웠다. 경과 시간만으론 계절마다 뜻이 달라 못 배웠고, 외면을 함께 넣자 맞았다: 두 달로 배워 남은 달 시험 — 기대 vs 실제 6월 26.3 vs 24.7대, 1월 16.4 vs 19.0, 3월 22.4 vs 21.7.
- **목록 안 순서도 좋아졌다**: 목록 위 20대 중 다음 사람도 반납한 비율 6월 51.5% → 56.0%, 1월 55.5 → 57.9, 3월 48.2 → 52.6 (세 달 모두 95% 범위가 0 위). 그래서 지금 목록은 모델 확률 순, '정비 먼저 볼 곳' 은 대여소마다 확률 합(기대 고장 수) 순이다.
- **목록 보장(분할 컨포멀 예측)**: "이 중 약 μ대가 진짜, 90% 확률로 최소 ⌊μ + q·σ⌋대". q 는 배울 때 안 쓴 달로 맞췄고(−2.34), 시험 달에서 하한이 맞은 비율 97.8~100%.
- **나무를 SQL 식으로 바꿔 클라우드 DB 가 5분마다 직접 계산**한다(서버 없음, sklearn 과 답 차이 0, 7특징 모두 DB = 파이썬 92/92 대). 경보가 울린 순간의 확률도 적어 두어 실시간으로 다시 채점한다.
- **다른 도시에서도 맞는다**: 서울로만 배운 모델을 대전 타슈(2025년 5·10월, 생년·성별 없음)에 그대로 쓰니 목록 위 10대 중 진짜 고장이 43.2 → 47.7%, 39.7 → 44.2%(두 달 모두 95% 범위가 0 위), '최소 몇 대' 보장은 100%·97.9% 맞았다. 기대 수는 조금 크게 잡았다(7.7 vs 6.7대) (`docs/model_daejeon.md`).
- **AI 가 본 이유(설명 가능한 AI)**: 자전거마다 특징 하나씩을 '목록의 보통 값' 으로 바꿨을 때 확률이 얼마나 변하는지로 가장 큰 이유 셋을 보여 준다(예: "서로 다른 17명이 연달아 반납 +32%p", "45분 전에 반납 +7%p"). 숫자만 보여 주면 믿기 어렵다.
- **스스로 배우기**: DB 가 3시간마다 지금 목록의 입력·확률을 적고, 다음 사람이 빌린 결과로 정답을 자동으로 채운다. 사람이 정답을 달지 않아도 학습 자료가 쌓이고, 주마다 지난 기록 + 실시간으로 다시 배운 모델이 **최근 자료에서 더 나을 때만** 바꾼다(`tools/retrain.py`). 첫 점검: 정답 172개에서 예측 35% vs 실제 34%.
- 함께 재 봤지만 쓰지 않은 것: 숨은 '고장/정상' 상태 모델(은닉 마르코프, 같은 수에서 이득 없음), 대여소 고장 경보(여러 자전거가 한꺼번에 헛대여한 대여소는 그 대여소 평소의 1.3~2배뿐, `docs/station_fault.md`).

6. 실시간으로는 되나 — 된다 (두 번 뒤집힌 결론)

처음 결론(틀렸음). 공개 실시간 API 중 대여소별 **자전거 대수**(OA-15493)만 보고, 과거 기록으로 30초마다 찍은 것처럼 만들어 "대수 −1 뒤 3분 안 +1" 로 헛대여를 잡아 봤다. 헛대여의 54% 만 보이고 그 깜빡임의 87% 는 남의 평범한 대여·반납이라 정밀도 0.2~1.7% — 쓸 수 없었다. 그래서 "하루 늦은 아침 목록" 으로 물러섰다(5-4).

다시 보니. 서울 열린데이터광장의 **대여이력 API(tbCycleRentData)** 는 자전거 번호·대여/반납 시각·대여소·거리·생년·성별을 **반납하자마자** 준다. 2026-09-25 10:59:44 에 조회했을 때 10:59:45 반납 기록이 이미 있었다(1초 차이는 시계 차). 같은 날(6/15)을 API 와 월별 파일로 받아 맞춰 보니 150,830행과 150,827행이 모두 짝을 이루고, 헛대여 4,085/4,087건, 그날 기록으로 만든 다음 날 아침 목록 79대가 같았다(`docs/api_parity.md`). 즉 **파일로 잰 모든 숫자(정밀도·선행 시간)가 실시간에도 그대로** 선다.

→ 클라우드 DB(Supabase) 가 5분마다 스스로(`supabase/live.sql` , pg_cron — 파이썬 엔진과 같은 결과를 검사로 확인) 기록을 받아 **서로 다른 두 번째 사람이 반납하는 그 순간** 경보를 켜고, 경보마다 "다음 다른 사람도 바로 반납했나" 를 스스로 채점한다. **운영 실측(2026-09-27 ~ 10-08)**: 경보 뒤 처음 빌린 다른 사람 3,478명 중 1,035명(29.8%)이 또 바로 반납 — 지난 기록으로 같은 방식으로 잰 32~34% 와 같은 흐름(평소 2.5%). 날짜별로는 결과가 100건 넘게 정해진 11일 모두 24~39%(공개 뷰 `ops_alarm_days` , 사이트·앱의 날짜별 막대). 5분 작업은 하루 288번 모두 성공(평균 19초, 10-07~08). 교훈: "공개 자료로는 안 된다" 는 결론을 내기 전에 **모든 공개 API 목록**을 먼저 봤어야 했다.

무료 DB 에서 버티게 (2026-10-02). 무료 클라우드 DB 는 메모리 0.5GB, 디스크 기본 250 IOPS 다. 5분 작업이 매번 후보 3천여 대의 7일 기록을 다시 계산하고 표 전체(110만 행)를 두세 번 훑고 있었는데, 그날 클라우드에서 측정까지 오래 하자 디스크 IO 예산이 바닥나 예약이 한 시간 넘게 시작조차 못 했다(`job startup timeout`). 작업을 다시 짰다: 경보(두 번째 사람)와 연쇄 2+ 는 **헛대여가 둘 이상 이어져야** 생기므로 '바로 앞 대여도 헛대여' 인 자전거만 후보로(3천여 → 3~5백 대, 답은 같음), 채점은 경보마다 다음 사람 한 줄만 색인으로, 자전거별 기록은 표 대신 덮개 색인에서, 서울 API 는 새로 붙은 쪽만 받는다. 옛·새 SQL 이 같은 답인지는 로컬 Postgres 에 6월 기록을 반납 순서대로 30분씩 흘려 넣으며 확인했고(36시간 73번 + 아침 목록 모두 같음), 5분 작업 한 번이 건드리는 서로 다른 페이지는 18,598쪽(145MB) → 5,171쪽(40MB)로 줄었다(`tools/live_sql_compare.py --cold`). 메모리 0.5GB 기계에서는 이 '작업 집합' 이 캐시 크기(224MB)에 견줘 작아야 디스크로 밀려나지 않는다. 다시 켠 뒤 클라우드의 5분 작업은 한 번 17~22초(전 평균 36.5초) — 단계마다 걸린 시간은 DB 가 스스로 `live.tick_log` 에 남긴다.

7. 서비스 — RIDEY (웹앱·아이폰·안드로이드)



같은 엔진이 파이썬(분석)·SQL(운영)·스위프트(아이폰)·자바스크립트(웹)에 있고, 서로 같은 답을 내는지 자동 검사한다. 비밀 키는 DB 금고(Vault)·키체인에만.

- **아침 목록**: 지도 + 정비 먼저 볼 대여소(누적 헛걸음 순) + 의심 자전거(노랑/빨강, 미신고 표시)
- **구별 보기·CSV**: 구를 고르면 그 구의 목록·지도·순위·동선만, 정비 담당에게 보낼 엑셀용 CSV 로 내려받기.
- **정비 동선**: 먼저 볼 대여소 10곳(또는 내 근처 10곳)을 도는 순서와 직선거리(가까운 곳부터 고른 뒤 꼬인 구간 풀기). 시연 날짜에는 "이 목록이 나온 뒤 처음 빌린 사람" 의 결과로 목록을 뒤돌아 채점해 보여 준다(6/15: 79명 중 30명, 38% 가 또 반납).
- **자전거 조회**: 빌리기 전 번호나 QR → "SPB-00000는 타지 마세요 — 서로 다른 28명이 빌리자마자 반납했어요" + 그 자전거의 '다음 사람도 반납할 확률'과 AI 가 본 이유. 아이폰은 **카메라로 번호판을 읽는다**(기기 안 글자 인식, 인터넷 없이) — 대여소의 자전거들을 비추면 의심 자전거에 빨간 상자. **내 대여소**(5곳까지, 폰에만 저장)를 넣어 두면 가기 전에 그곳에 서 있는 '피할 번호' 가 보인다.
- **구조대**: 가까운 의심 자전거부터(위치를 켜면 거리순) 3초 보고 "체인·타이어·안장·멀쩡함" 한 번 탭 → 서버에 모여, 사람이 고장이라고 확인한 대여소가 정비 순위 맨 위로 올라간다. 경보 뒤 다음 대여까지 중앙값 30분, 그 사이 같은 대여소 이용자 중앙값 2명 — 경보의 65% 는 확인해 줄 사람이 있다.
- **현장 조사**: '오늘 갈 곳 — 눈 가리고' 가 가까운 경보 대여소 3곳과 그 근처 경보 없는 대여소 2곳을 섞어 이름만 보여 준다(어느 쪽인지 모르게 — 선입견 없이). 그곳 자전거를 한 대씩 보고 상태·메모·사진(폰에서 줄여 보냄)을 남긴다. 서버에 안 닿으면 폰에 모았다가 다음에 보낸다. 그달 기록이 공개되면 엔진 판단과 맞춰 95% 신뢰구간과 함께 정리 (`analysis/field_validation.py`).
- **시연**: 2026-06-15 하루를 재생 — 경보가 켜지고, 막을 수 있던 헛걸음이 쌓이고, 뒤늦은 고장 신고가 들어온다.

8. 이미 있는 것과의 차이 (`docs/related_work.md`)

- 연구: 대여기록으로 고장 자전거를 추정하는 연구가 있다 — Kaspi 등 2016(다른 자전거는 빌려 가는데 혼자 남는 '외면' 을 베이즈 모델로), Alhussam 등 2024(은닉 마르코프 모델), 2025 자기지도 트랜스포머(**GPS 궤적과 회사가 준 고장 정답**이 필요), 바르셀로나 2024(**정비 기록**으로 생존 분석). 우리 자료로 '외면' 신호도 재 봤다: 반납 뒤 같은 대여

소에서 160번 넘게 외면받은 자전거는 그사이 고장신고가 약 100배 — 신호는 있지만 혼자서는 약하다(다음 헛대여 6%).
- 따릉이 앱: 사람이 신고하는 버튼만 있다.
- 우리: ① 한국 공개 자료로 누구나 재현 ② **서로 다른 사람** 연쇄(같은 사람 재시도 제거) ③ 기준을 한 달로 정하고 다른 달·도시에서 검증 ④ **이용자 화면에 근거를 그대로**(앞사람 몇 명이 포기했는지) ⑤ 공개 실시간 자료의 한계를 측정해 운영기관 제안으로 연결.

9. 한계

- "고장" 의 정답 자료가 없다. 고장 신고는 늦고 드물어 정답으로 쓰면 정밀도가 낮게 나온다 → 다음 사람의 헛걸음을 주 지표로 썼다. **현장 확인(Phase 5)** 으로 보완할 것.
- 같은 사람 판정은 생년·성별로 한 근사다(대전은 불가). 생년·성별이 없는 자료에선 **반납 2분 안 재대여를 같은 사람으로** 보면 된다 — 1월로 정해 3·6월에 시험한 결과 정밀도 차이 0.2%p 이하, 경보 수 거의 같음. 아예 안 거르면 정밀도 −4~5%p, 경보 +25~28% (`docs/no_who.md`).
- 실시간 자료는 늘 고르게 오지 않는다. 2026-09-25(추석 당일) 14시부터 대여이력 API 가 한 시간에 평소의 2~5%만 내놓았고(비 없음), 실시간 대여소 현황도 5분 동안 1,000곳 중 42곳만 바뀌었다 — 자료가 늦은 건지 이용이 실제로 멈춘 건지는 확인하지 못했다. 그래서 시간마다 '지난 7일 같은 시각 중 가장 한산했던 날의 30%' 미만이면 모자란 시간으로 보고, 앱에 알리고, 48시간 동안 다시 받고, 그 시간이 낀 경보는 채점을 보류한다.

10. 사회적 효과와 확장

- 서울에서만 1년 약 5만 번의 헛걸음, 그리고 신고되지 않은 채 굴러다니는 고장 자전거를 줄인다.
- 운영기관 제안서 초안: `docs/proposal.md` (자전거별 기록을 1시간 단위로만 열어도 아침 목록을 하루 여러 번 갱신).
- 공공자전거가 있는 다른 도시(대전 타슈 검증 완료, 세종 어울링, 창원 누비자)에 그대로 적용.

코드·자료 재현: README.md · 만든 날 2026-11-05